

LAPORAN PROYEK ANTREAN TIKET BIOSKOP ALGORITMA & STRUKTUR DATA

Disusun untuk memenuhi tugas akhir Project-Based Learning
(PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)



Disusun Oleh :

1. M. Lutfi Ramadhan Warendra (J0403251040)
2. Fateeh Falah Hendarto (J0403251070)
3. Wildhan Dzikrihantara (J0403251098)

**Program Studi Teknologi Rekayasa Perangkat Lunak
Sekolah Vokasi IPB University**

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa atas segala limpahan kasih, karunia, dan kehendak-Nya sehingga Tugas Akhir Project Base Learning (PBL) Mata Kuliah Algoritma & Struktur Data (TPL2106) dapat diselesaikan dengan baik. Selesaiannya Tugas Akhir Project Base Learning (PBL) Mata Kuliah Algoritma & Struktur Data (TPL2106) ini tidak lepas dari bantuan, bimbingan, dan doa dari beberapa pihak. Oleh karena itu, penulis mengucapkan terima kasih sebesar-besarnya kepada yang terhormat:

1. Ibu Dr. Shelve Nidya Neyman S.Kom, M.Si. selaku dosen pengampu kuliah Algoritma & Struktur Data (TPL2106).
2. Ibu Dr. Sofiyanti Indriasari, S.Kom., M.Kom. selaku dosen pengampu praktikum Algoritma & Struktur Data (TPL2106).
3. Tim Asisten Dosen Mata Kuliah Algoritma & Struktur Data (TPL2106).

Dalam penyusunan makalah ini, ditulis berdasarkan proyek kami yang berjudul “Bioskop CACB: Sistem Antrean Tiket Bioskop”. Penulis menyadari bahwa Tugas ini masih kurang sempurna. Untuk itu penulis mengharapkan kritik dan saran yang bersifat membangun kesempurnaannya.

Akhir kata, penulis berharap agar hasil Project Base Learning ini dapat bermanfaat untuk pembaca.

Bogor, 24 Juni 2026

Kelompok 4

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I.	
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Proyek.....	1
1.3 Ruang Lingkup.....	2
BAB II. ANALISIS DAN PERANCANGAN.....	4
2.1 Deskripsi Sistem.....	4
2.2 Struktur Data yang Digunakan.....	4
2.2.1 Linked List.....	5
2.2.2 Queue.....	6
2.2.3 Dictionary.....	6
2.3 Algoritma yang Digunakan.....	7
2.3.1 Searching.....	7
2.3.2 Sorting.....	7
2.4 Diagram Alir Program (Flowchart).....	9
2.4.1 Diagram Flow Penggunaan Umum.....	9
2.4.2 Diagram Flow Aplikasi.....	10
2.5 Struktur Program.....	10
BAB III. IMPLEMENTASI PROGRAM.....	13
3.1 Tampilan Menu Utama.....	13
3.2 Fitur Create.....	13
3.3 Fitur Read.....	14
3.3.1 Pilih Film.....	14
3.3.2 Lihat Antrean.....	15
3.3.3 Lihat Data Pemesanan.....	16
3.3.4 Fitur Backend: Load Queue (Tidak memiliki interface).....	16
3.3.5 Fitur Backend: Load Tickets (Tidak memiliki interface).....	17
3.4 Fitur Update.....	18
3.5 Fitur Delete.....	19
3.5.1 Hapus Delete (Delete Film).....	19
3.5.2 Batalkan Antrean (Cancel Queue).....	19
3.5.3 Batalkan Pemesanan (Cancel Order).....	19
3.5.4 Reset Antrean & Pemesanan.....	20
3.6 Fitur Searching.....	20
3.6.1 Cari Data Pemesanan (searchTicket).....	20
3.6.2 Cari Duplikasi Pemesanan (searchTempLogPemesananDuplicates).....	21
3.6.3 Cari Data Database (searchData).....	21

3.6.4 Kalkulasi Nama di Antrean (countNameInQueue).....	21
3.7 Fitur Sorting.....	22
3.7.1 Pengurutan Daftar Kursi (Seat Sorting).....	22
3.8 Penyimpanan Data (File Handling).....	22
3.8.1 Format File yang Digunakan.....	22
3.8.2 Proses Pembacaan File.....	23
3.8.3 Proses Penyimpanan File.....	23
3.8.4 Tabel Format Penyimpanan dan Contoh.....	24
3.9 Validasi Input dan Error Handling.....	24
BAB IV. PENGUJIAN & ANALISIS.....	26
4.1 Skenario Pengujian.....	26
4.2 Kelebihan Sistem.....	27
4.3 Keterbatasan Sistem.....	27
BAB V. KENDALA & SOLUSI.....	29
BAB VI. PEMBAGIAN TUGAS ANGGOTA.....	31
6.1 Fateeh Falaah Hendharto (J0403251098).....	31
6.2 Muhammad Lutfi Ramadhan Warendra (J0403251040).....	31
6.3 Wildhan Dzikrihantara (J0403251098).....	31
BAB VII.	
KESIMPULAN & SARAN.....	33
7.1 Kesimpulan.....	33
7.2 Saran.....	33
DAFTAR PUSTAKA.....	35
LAMPIRAN.....	36
A. Link Github.....	36
B. Screenshoot Program.....	36
C. Struktur Proyek (Folder).....	40
D. Commit History Github.....	40
E. Source Code Utama (main.py & antrean.py).....	40

BAB I.

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong berbagai sektor layanan untuk beralih dari proses manual menuju sistem yang lebih terstruktur, cepat, dan akurat. Dalam konteks layanan pemesanan tiket bioskop, proses antrean yang masih dilakukan secara konvensional sering menimbulkan ketidakteraturan urutan pelayanan, memperlambat kerja kasir, serta menyulitkan pencatatan riwayat pemesanan.

Struktur data *Queue* merupakan solusi yang relevan karena menerapkan prinsip *First In First Out* (FIFO), yaitu elemen yang masuk terlebih dahulu akan diproses terlebih dahulu. Karakteristik ini sangat sesuai untuk sistem antrean tiket bioskop, di mana pelanggan yang datang lebih awal harus dilayani lebih dahulu (Cormen et al., 2022). Selain itu, *Linked List* dapat dimanfaatkan untuk menyimpan data riwayat pemesanan secara dinamis karena struktur ini lebih fleksibel dalam penambahan dan penghapusan elemen dibandingkan struktur data statis (Goodrich et al., 2013). Dengan demikian, kombinasi *Queue* dan *Linked List* dapat membantu meningkatkan efisiensi pengelolaan antrean dan mendukung pencatatan data yang berkelanjutan.

1.2 Tujuan Proyek

Tujuan dari pengembangan proyek **BIOSKOP CACB** adalah menghasilkan sebuah aplikasi manajemen pemesanan tiket bioskop berbasis *Command Line Interface* (CLI) yang interaktif dan mudah digunakan dengan bahasa pemrograman Python. Aplikasi ini dirancang untuk membantu proses pelayanan kasir agar lebih terstruktur, cepat, dan terdokumentasi dengan baik.

Secara khusus, proyek ini bertujuan untuk:

1. Membuat aplikasi manajemen pemesanan tiket bioskop berbasis terminal (*CLI*) interaktif menggunakan bahasa Python.
2. Mengimplementasikan struktur data *Queue* untuk mengatur logika antrean dengan prinsip *First In First Out* (FIFO) dan *Linked List* untuk menyimpan riwayat pemesanan secara dinamis.
3. Menerapkan konsep *file handling* menggunakan file TXT agar data film, antrean, dan riwayat pesanan dapat disimpan secara permanen dan dapat diproses kembali saat aplikasi dijalankan.
4. Memberikan solusi yang efisien bagi kasir dalam melayani pelanggan, mengelola transaksi, serta mencetak laporan penjualan tiket secara terstruktur.

1.3 Ruang Lingkup

Ruang lingkup pengembangan proyek **BIOSKOP CACB** dibatasi pada pembuatan sistem pemesanan tiket bioskop berbasis *Command Line Interface* dengan bahasa pemrograman Python. Sistem ini menggunakan media penyimpanan data berbasis TXT untuk mendukung kebutuhan *file handling* agar data dapat disimpan, dibaca, dan diperbarui secara berkelanjutan (Python Software Foundation, n.d.-a, n.d.-b).

Adapun ruang lingkup fitur yang dikembangkan adalah sebagai berikut:

1. CRUD (*Create, Read, Update, Delete*) untuk data film.
2. Manajemen antrean yang mencakup operasi *enqueue*, *dequeue*, dan pembatalan antrean.
3. *Searching* untuk pencarian riwayat data pemesanan pelanggan.
4. *Sorting* untuk pengurutan nomor kursi tiket.
5. Fitur ekspor *invoice* dan laporan ke dalam format PDF.

Sistem ini difokuskan pada pengelolaan lokal dan tidak mencakup antarmuka grafis maupun integrasi daring. Dengan pembatasan tersebut, aplikasi diharapkan mampu menjadi solusi praktis yang sederhana, efisien, dan sesuai untuk kebutuhan operasional kasir bioskop.

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Bioskop CACB: Sistem Antrean Tiket Bioskop merupakan aplikasi berbasis *Command Line Interface* (CLI) yang dibangun menggunakan bahasa pemrograman Python untuk memudahkan kasir dalam mengelola data film dan antrean pelanggan. Melalui sistem ini, kasir dapat melakukan pengelolaan data film berupa penambahan, perubahan, dan penghapusan data, serta mengatur alur pelayanan pelanggan yang ingin memesan tiket secara terstruktur. Karakteristik berbasis CLI menjadikan sistem ini sederhana, ringan, dan sesuai digunakan pada lingkungan operasional yang membutuhkan interaksi cepat namun tetap fungsional.

Secara arsitektural, sistem menerapkan teknik *File Handling* sebagai mekanisme penyimpanan data persisten. Seluruh data penting, seperti log pemesanan, ketersediaan kursi, daftar antrean, dan daftar film, disimpan ke dalam file teks berformat *comma-separated* (TXT/CSV). Dengan pendekatan ini, data tidak hanya tersimpan sementara di memori, tetapi tetap bertahan meskipun program ditutup dan dijalankan kembali. Mekanisme ini menjadi penting karena sistem antrean memerlukan kesinambungan data agar informasi yang tersimpan tetap konsisten pada setiap sesi penggunaan.

2.2 Struktur Data yang Digunakan

Dalam perancangan sistem, digunakan beberapa struktur data yang disesuaikan dengan kebutuhan pengolahan data pada masing-masing modul. Pemilihan struktur data dilakukan untuk mendukung efisiensi proses penyimpanan, pencarian, dan pengelolaan antrean secara keseluruhan.

2.2.1 Linked List

Struktur data *Linked List* digunakan pada class Ticket yang terdapat pada file ticket.py untuk menyimpan record data pemesanan tiket atau riwayat transaksi. Pemilihan *Linked List* didasarkan pada sifatnya yang dinamis, sehingga penambahan dan penghapusan record pemesanan dapat dilakukan tanpa harus menyiapkan alokasi memori berdekatan sebagaimana pada array. Dalam konteks riwayat pemesanan, kebutuhan penyimpanan data cenderung bertambah seiring penggunaan sistem, sehingga struktur dinamis lebih sesuai dibandingkan struktur statis. Selain itu, *Linked List* memudahkan proses manipulasi data karena setiap node dapat dihubungkan secara berurutan dan diakses melalui penelusuran dari node awal.

2.2.2 Queue

Struktur data *Queue* digunakan pada class *Queue* yang terdapat pada file *queue.py* untuk mengelola antrean pelanggan di depan kasir. Pemilihan *Queue* sangat relevan karena struktur ini mengikuti prinsip *First-In-First-Out* (FIFO), yaitu pelanggan yang datang lebih dahulu akan dilayani terlebih dahulu. Prinsip ini merepresentasikan logika antrean nyata di dunia pelayanan, sehingga implementasinya selaras dengan kebutuhan sistem pemesanan tiket bioskop. Dengan menerapkan *Queue*, sistem dapat menjaga urutan pelayanan secara adil dan konsisten, sekaligus mempermudah proses penambahan pelanggan baru dan pengambilan pelanggan yang telah dilayani.

2.2.3 Dictionary

Struktur data *Dictionary* digunakan pada modul *dataOps.py* saat memuat data dari file database *.txt* ke memori utama (*RAM*). *Dictionary* dipilih karena mampu menyimpan pasangan *key-value* sehingga sangat efektif untuk proses *caching* dan pencarian data berbasis identitas tertentu, misalnya *id_film* atau *id_log*. Secara umum, operasi pencarian berdasarkan kunci pada *Dictionary* memiliki kompleksitas rata-rata yang sangat efisien, yaitu mendekati $O(1)$, sehingga cocok untuk meningkatkan kecepatan akses data pada aplikasi yang memerlukan pembacaan berulang. Hal ini mendukung performa sistem ketika data film maupun data pemesanan harus diakses secara cepat dan berulang selama proses operasional berlangsung.

Struktur Data	Fungsi
Linked List	Menyimpan record data pemesanan tiket atau riwayat transaksi secara dinamis.
Queue	Mengelola antrean pelanggan dengan prinsip FIFO (First-In-First-Out).
Dictionary	Memuat data dari file ke RAM untuk pencarian berbasis key-value yang efisien.

2.3 Algoritma yang Digunakan

2.3.1 Searching

Algoritma *Searching* pada sistem ini menggunakan metode *Sequential Search* atau *Linear Search* untuk mencari data pemesanan tiket berdasarkan nama pelanggan. Proses pencarian dilakukan dengan menelusuri *Linked List* dari node pertama hingga data yang sesuai ditemukan atau seluruh data telah diperiksa. Perbandingan nama dilakukan tanpa membedakan huruf besar dan huruf kecil agar pencarian lebih fleksibel. Jika data ditemukan, sistem akan menampilkan detail pemesanan kepada pengguna.

Metode ini sesuai digunakan pada struktur *Linked List* karena data tidak tersimpan dalam indeks tertentu seperti pada array, sehingga pencarian harus dilakukan secara berurutan dari awal hingga akhir. Dengan cara ini, fungsi `searchTicket` dapat bekerja secara sederhana, efektif, dan sesuai dengan karakteristik struktur data yang digunakan.

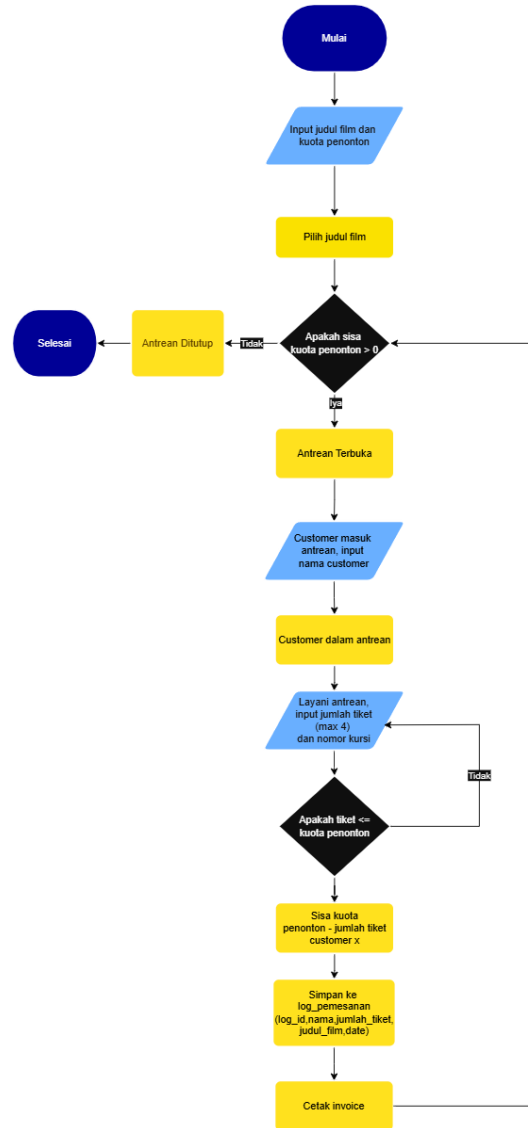
2.3.2 Sorting

Algoritma *Sorting* yang digunakan dalam sistem adalah algoritma bawaan Python yang digunakan oleh fungsi `sorted()` dan

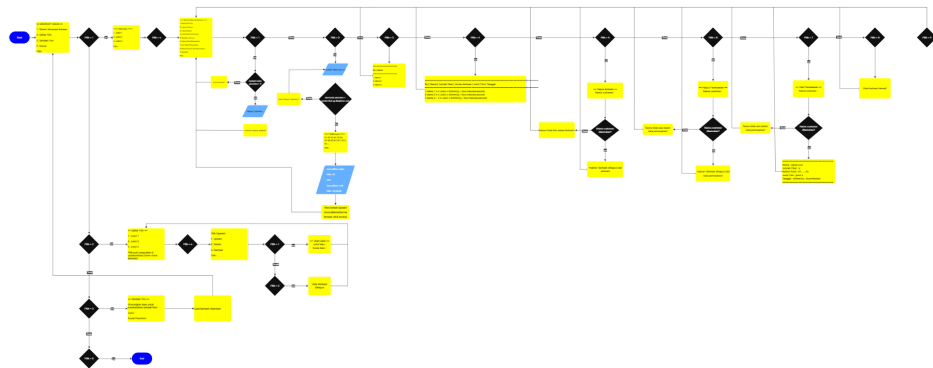
`list.sort()`. Pada modul `seatSort.py`, algoritma ini dimanfaatkan dengan ekstraksi *key* kustom agar daftar string nomor kursi dapat diurutkan secara numerik. Misalnya, data kursi seperti K1, K10, dan K2 dapat ditata kembali menjadi urutan K1, K2, K10 dengan memisahkan bagian huruf dan angka, lalu mengurutkan berdasarkan nilai integer-nya. Timsort dipilih karena bersifat stabil, adaptif, dan memiliki kompleksitas waktu yang efisien, yaitu $O(N \log N)$, sehingga sangat sesuai untuk pengurutan data pada sistem yang memerlukan hasil konsisten dan cepat.

2.4 Diagram Alir Program (Flowchart)

2.4.1 Diagram Flow Penggunaan Umum



2.4.2 Diagram Flow Aplikasi



Sumber:

- [Sumber Diagram](#)
- https://miro.com/app/board/uXjVGD2OHVo=/?share_link_id=12187728476

2.5 Struktur Program

Arsitektur program pada sistem ini disusun secara modular agar setiap komponen memiliki tanggung jawab yang jelas dan mudah dipelihara. Pembagian file dilakukan untuk memisahkan logika utama, struktur data, antarmuka pengguna, serta pengelolaan data persistennya. Dengan pendekatan modular, pengembangan, pengujian, dan pemeliharaan kode menjadi lebih terstruktur.

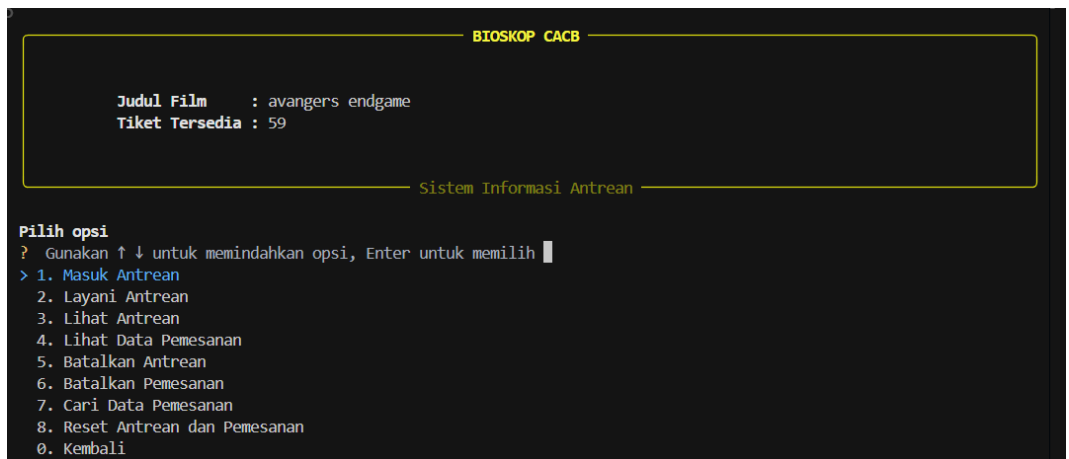
1. File `main.py` berperan sebagai titik masuk utama (entry point) aplikasi yang memuat antarmuka navigasi Main Menu. File ini menjadi pusat penghubung antarfitur sehingga pengguna dapat memilih modul yang ingin dijalankan.
2. Folder `CRUD/operations/antrean.py` berisi logika inti manajemen sistem antrean dan alur pemesanan, meliputi proses masuk antrean, melayani antrean, dan membatalkan antrean. Modul ini menjadi inti operasional sistem karena menangani perpindahan status pelanggan dari antrean ke proses transaksi.

3. File `CRUD/operations/queue.py` berisi implementasi class struktur data Queue beserta algoritmanya. Modul ini bertanggung jawab terhadap operasi dasar seperti penambahan data ke belakang antrean dan pengambilan data dari depan antrean.
4. File `CRUD/operations/ticket.py` berisi implementasi class struktur data Linked List beserta algoritmanya. Modul ini digunakan untuk menyimpan riwayat transaksi atau data pemesanan tiket secara dinamis.
5. File `CRUD/operations/film.py` menangani logika fungsi CRUD untuk data film bioskop. Modul ini mengelola seluruh proses pengubahan informasi film yang tersedia dalam sistem.
6. File `CRUD/operations/order.py` mengatur antarmuka interaktif ketika pelanggan memilih kursi, termasuk tampilan denah kursi dan proses pemilihan nomor kursi.
7. File `CRUD/operations/report.py` dan `invoice.py` merupakan modul yang mengimplementasikan pustaka `reportlab` untuk mencetak laporan penjualan dan struk tiket dalam format PDF. Modul ini mendukung kebutuhan dokumentasi transaksi secara profesional.
8. File `CRUD/utls/dataOps.py` berfungsi sebagai modul bantu untuk menangani file handling, baik saat membaca data (`getAllData`) maupun saat menulis atau memperbarui data (`updateData`) ke file TXT. Modul ini memastikan data tetap persisten dan sinkron dengan kondisi terbaru.
9. File `CRUD/utls/seatSort.py` berisi algoritma pengurutan nomor kursi agar dapat ditampilkan dalam urutan numerik yang benar. Modul ini digunakan untuk memastikan representasi data kursi tetap rapi dan mudah dibaca.
10. File `CRUD/utls/node.py` berisi representasi class dasar Node yang digunakan bersama oleh Queue dan Linked List. Class ini menjadi elemen fundamental dalam penyusunan struktur data berbasis tautan.
11. Folder `CRUD/data/` digunakan sebagai direktori penyimpanan file-file persisten berformat `.txt` yang berperan sebagai database lokal aplikasi. Folder ini menjadi tempat penyimpanan utama seluruh data operasional sistem.

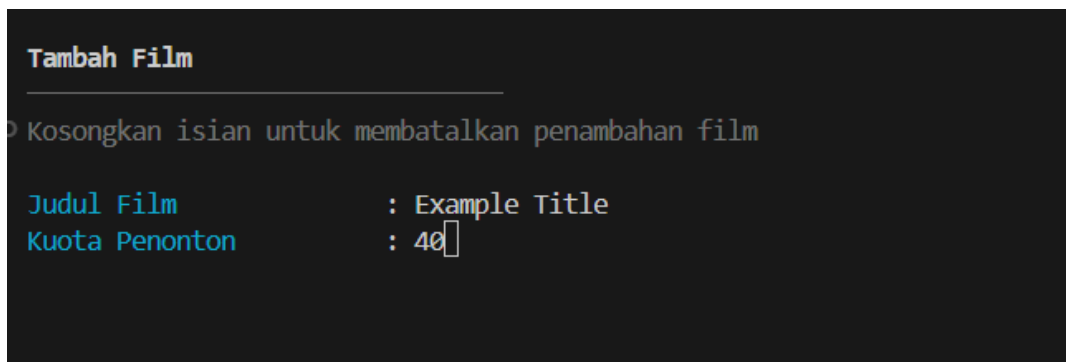
```
antrean-tiket-bioskop/
├── .gitignore
├── CRUD/
│   ├── data/
│   │   ├── .gitkeep
│   │   ├── data_antrean.txt
│   │   ├── data_film.txt
│   │   ├── log_pemesanan.txt
│   │   ├── temp_log_pemesanan.txt
│   │   └── temp_seat.txt
│   ├── history/
│   │   ├── invoices/
│   │   │   └── .gitkeep
│   │   └── reports/
│   │       └── .gitkeep
│   ├── lib/
│   ├── operations/
│   │   ├── antrean.py
│   │   ├── film.py
│   │   ├── invoice.py
│   │   ├── order.py
│   │   ├── queue.py
│   │   ├── report.py
│   │   └── ticket.py
│   └── utils/
│       ├── clear.py
│       ├── dataOps.py
│       ├── deleteTempSessions.py
│       ├── idGenerator.py
│       ├── loading.py
│       ├── node.py
│       └── seatSort.py
├── README.md
├── main.py
└── requirements.txt
```


BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama



3.2 Fitur Create



Fitur Tambah Film pada aplikasi ini diimplementasikan melalui pemisahan tugas antara antarmuka pengguna (main.py) dan manajemen data (film.py). Pada main.py, sistem meminta pengguna memasukkan judul film dan kuota penonton dengan melakukan validasi awal, seperti mencegah penggunaan tanda koma pada judul agar tidak merusak format database serta memastikan kuota berupa angka valid dengan batas maksimal 60 orang. Setelah input tervalidasi, data dikirim ke fungsi addFilm() pada film.py, di mana sistem akan memuat data eksisting dan melakukan pengecekan duplikasi untuk memastikan judul film yang dimasukkan belum terdaftar. Jika validasi kelayakan ini lulus, sistem akan secara otomatis menghasilkan ID unik baru (generate ID), menyisipkan rincian film tersebut ke dalam dictionary penyimpanan, dan diakhiri dengan memperbarui file penyimpanan utama agar data tersimpan secara permanen.

3.3 Fitur Read

3.3.1 Pilih Film

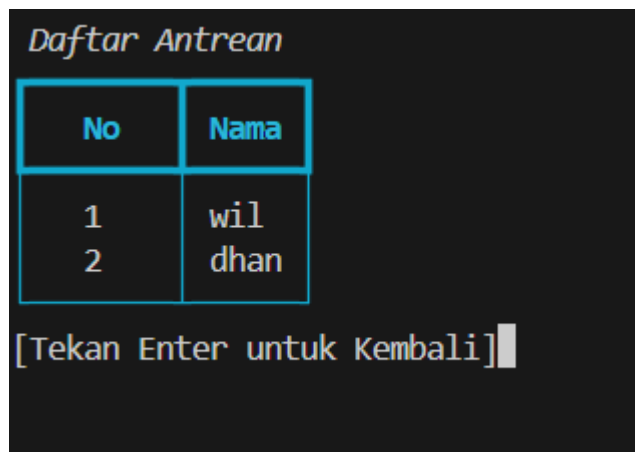
```
Pilih Film
Pilih untuk melanjutkan.

? Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih
> avangers endgame [60 kuota]
  Example Title [40 kuota]
  --- Keluar ---
```

Fitur Pemilihan Film diimplementasikan untuk menjembatani interaksi antara pengguna dengan basis data film yang tersedia di dalam sistem. Secara teknis, fungsi ini bekerja dengan melakukan pembacaan menyeluruh terhadap berkas penyimpanan JSON/teks yang menyimpan data operasional film. Setelah data berhasil dimuat ke dalam memori utama, algoritma akan melakukan proses iterasi untuk mengekstrak setiap entitas

judul film beserta informasi ketersediaan kuotanya. Kumpulan entitas tersebut kemudian direstrukturisasi menjadi daftar pilihan dinamis yang terintegrasi dengan pustaka antarmuka terminal interaktif. Pengguna kemudian dapat menavigasikan daftar tersebut secara langsung, di mana fungsi ini pada akhirnya akan memvalidasi masukan dan mengembalikan pengenalan unik (ID) dari film yang terpilih sebagai kunci parameter untuk memicu alur eksekusi proses sistem berikutnya.

3.3.2 Lihat Antrean



No	Nama
1	wil
2	dhan

[Tekan Enter untuk Kembali]

Fitur Lihat Antrean dirancang untuk memvisualisasikan status antrean pelanggan yang sedang berlangsung namun belum mendapatkan pelayanan tiket. Implementasi fungsionalitas ini memanfaatkan arsitektur struktur data antrean (Queue) yang dibangun di atas fondasi Linked List. Ketika fitur ini dieksekusi, algoritma program akan terlebih dahulu memvalidasi ketersediaan antrean. Jika antrean tidak kosong, sistem melakukan penelusuran beruntun (traversal) yang diinisiasi dari simpul terdepan (pointer front) hingga mencapai simpul paling akhir (pointer rear). Selama proses iterasi berlangsung pada struktur data tersebut, atribut spesifik seperti nomor urut dan identitas pelanggan diekstrak dari setiap simpul, lalu diformat secara sistematis untuk dicetak

menjadi antarmuka tabel visual pada layar terminal, sehingga memudahkan administrator dalam memantau operasional antrian secara langsung.

3.3.3 Lihat Data Pemesanan

Daftar Pemesanan Tiket				
No	Nama	Jumlah Tiket	Nomor Kursi	Tanggal
1	wil	1	K1	24/06/26 - 22:15:46
2	teh	2	K16 K17	24/06/26 - 22:16:09

[Tekan Enter untuk Kembali]

Fitur Lihat Data Pemesanan dikembangkan untuk memberikan pelaporan sekilas terkait rincian transaksi tiket yang telah berhasil diverifikasi dan dilayani di dalam sistem. Secara algoritmis, pencatatan data pemesanan dioperasikan murni menggunakan struktur data Linked List tunggal. Proses pembacaan data dilakukan melalui mekanisme iterasi traversal yang dimulai dari simpul awal (pointer head). Apabila struktur data tidak dalam keadaan kosong, sistem akan melangkah secara sekuensial menelusuri rantai simpul yang ada. Pada setiap lompatan, informasi krusial seperti nama pembeli, kuantitas tiket, alokasi nomor kursi, dan stempel waktu transaksi akan diakuisisi untuk kemudian dikonstruksi secara berurutan menjadi luaran berupa tabel data yang komprehensif pada antarmuka baris perintah pengguna.

3.3.4 Fitur Backend: Load Queue (Tidak memiliki interface)

Fungsi muat ulang antrian memegang peranan vital sebagai mekanisme pemulihan status (state restoration) untuk menjaga kegigihan data (data persistency) operasional antrian antar sesi eksekusi program. Pendekatan logis dari fungsi ini didasarkan pada proses deserialisasi rekaman data berformat dictionary dari media

penyimpanan eksternal kembali ke dalam memori utama komputer. Ketika manajemen antrian untuk suatu judul film diakses, program secara otomatis mengekstraksi riwayat simpul antrian yang tertaut pada pengenalan film terkait. Sistem selanjutnya melakukan rekonstruksi struktur Linked List secara bottom-up dengan menginisiasi ulang objek-objek simpul (nodes), memasukkan properti data lampau ke dalamnya, dan merajut kembali tautan arah (pointers) antar simpul. Proses ini memastikan bahwa hierarki dan urutan Queue di dalam memori tetap utuh dan konsisten dengan kondisi terakhir sebelum aplikasi dihentikan.

3.3.5 Fitur Backend: Load Tickets (Tidak memiliki interface)

Fungsi muat ulang tiket diimplementasikan untuk melakukan sinkronisasi riwayat data pemesanan tiket dari berkas log penyimpanan sementara agar dapat dimuat kembali ke dalam struktur memori aktif program. Mekanisme fungsional ini berpusat pada proses penyaringan dan pemulihan rekaman persisten. Secara sekuensial, sistem akan membaca keseluruhan jejak transaksi dari database, kemudian memfilter entitas data tersebut secara ketat berdasarkan kecocokan nilai parameter judul film yang sedang aktif dioperasikan. Terhadap setiap entitas data yang lolos seleksi, sistem memformat ulang validitas tipe data alokasi kursi, untuk selanjutnya mengeksekusi metode penambahan simpul (addTicket) secara berulang. Alur kerja ini meregenerasi struktur Linked List pemesanan historis secara presisi, sehingga memungkinkan aplikasi untuk mengkalkulasi ulang rekonsiliasi data ketersediaan kursi tanpa kehilangan rekam jejak transaksi sesi sebelumnya.

3.4 Fitur Update

```
Pilih Film
Pilih untuk melanjutkan.
? avangers endgame [60 kuota]
? 1. Update

Ubah Film
-----

Judul Film      : avangers endgame
Kuota Penonton  : 60

Kosongkan isian jika tidak ingin mengganti data

Judul Film Baru : the game changer
Kuota Baru      : 40
```

Fitur Ubah Data Film dikembangkan untuk memodifikasi rincian data film yang telah terdaftar di sistem, seperti memperbarui judul maupun menyesuaikan batas kuota penonton. Secara teknis, proses ini diawali dengan memuat data film dari berkas penyimpanan ke memori menggunakan struktur data Dictionary. Saat pengguna memasukkan perubahan, sistem akan memvalidasi input untuk mencegah duplikasi judul maupun format karakter yang tidak sah. Apabila data baru dinyatakan valid—atau dikosongkan untuk mempertahankan data lama—sistem akan langsung memperbarui atribut pada target Dictionary tersebut. Terakhir, keseluruhan data yang telah diubah akan disimpan kembali ke dalam berkas utama agar perubahan tersimpan secara permanen.

3.5 Fitur Delete

3.5.1 Hapus Delete (Delete Film)

```
Pilih Film
Pilih untuk melanjutkan.

? Example Title [40 kuota]
? Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih █
1. Update
> 2. Delete
0. Kembali
```

Fitur ini berfungsi untuk menghapus entri tayangan film dari sistem secara permanen. Secara teknis, sistem akan menghapus data film dari memori (Dictionary) berdasarkan ID uniknya. Bersamaan dengan itu, algoritma juga akan membersihkan seluruh sesi antrean dan riwayat pemesanan tiket yang terkait dengan judul film tersebut, baik dari memori maupun dari berkas penyimpanan lokal, agar tidak meninggalkan sisa data.

3.5.2 Batalkan Antrean (Cancel Queue)

```
Nama customer yang akan dibatalkan (Enter untuk kembali): █
```

Fitur ini digunakan untuk menghapus pelanggan dari daftar tunggu sebelum tiket mereka diproses. Algoritmanya bekerja dengan menelusuri nama pelanggan pada struktur data antrean (Queue). Jika ditemukan, simpul (node) pelanggan tersebut akan diputus dari rantai antrean. Setelah itu, sistem akan melakukan iterasi lanjutan untuk mengkalkulasi ulang dan menyesuaikan (menurunkan) nomor urut semua pelanggan lain yang berada di belakangnya.

3.5.3 Batalkan Pemesanan (Cancel Order)

```
Nama customer yang akan dibatalkan (Enter untuk kembali): █
```

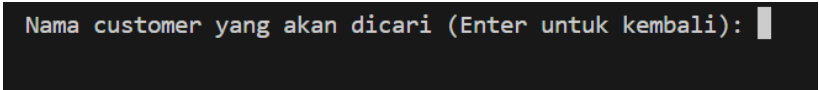
Fitur ini memungkinkan pembatalan transaksi tiket yang sebelumnya telah berhasil diverifikasi. Sistem akan menelusuri struktur data Linked List pemesanan menggunakan kombinasi parameter khusus (nama, jumlah tiket, dan nomor kursi) untuk menghindari salah hapus pada nama yang sama. Setelah data pemesanan dihapus dari Linked List dan berkas riwayat (log), sistem akan secara otomatis mengembalikan (refund) status ketersediaan kursi beserta kuota tiket ke kondisi semula.

3.5.4 Reset Antrean & Pemesanan

Fitur ini berfungsi sebagai mekanisme pembersihan masal untuk mereset sesi pada suatu film. Secara programatis, sistem akan memutuskan pointer awal (front/head) dan akhir (rear/tail) pada struktur data Queue maupun Linked List lalu mengatur nilainya menjadi kosong (None). Hal ini secara instan akan mengosongkan seluruh antrean dan pemesanan aktif di dalam memori untuk film tersebut.

3.6 Fitur Searching

3.6.1 Cari Data Pemesanan (searchTicket)



```
Nama customer yang akan dicari (Enter untuk kembali):
```

Fitur Cari Data Pemesanan dirancang untuk menelusuri dan menemukan rincian transaksi tiket spesifik berdasarkan nama pelanggan. Secara algoritmis, sistem mengimplementasikan metode Sequential Search (Linear Search), di mana penelusuran dieksekusi secara iteratif dari simpul pertama (head) pada Linked List hingga data yang relevan ditemukan atau seluruh simpul selesai dievaluasi. Guna memberikan fleksibilitas dan meminimalisasi galat akibat input pengguna, komparasi dilakukan secara case-insensitive (tanpa membedakan huruf kapital dan

kecil). Pendekatan pencarian linear ini merupakan metode alamiah yang paling sesuai dengan karakteristik Linked List. Berbeda dengan struktur Array yang elemennya berindeks, elemen pada Linked List hanya dapat diakses dengan mengikuti alur pointer dari satu simpul ke simpul selanjutnya sehingga penelusuran mutlak harus dilakukan secara berurutan. Saat data ditemukan, sistem akan merender detail pemesanan pada antarmuka pengguna.

3.6.2 Cari Duplikasi Pemesanan (searchTempLogPemesananDuplicates)

Fitur Cari Duplikasi Pemesanan dikembangkan khusus untuk menelusuri log transaksi persisten guna mengidentifikasi seluruh pemesanan yang menggunakan komposit nama pelanggan dan judul film yang identik. Algoritma ini beroperasi dengan memuat keseluruhan data log ke dalam struktur Dictionary, kemudian melakukan proses pemfilteran secara menyeluruh (exhaustive search). Pada tiap iterasi, atribut dievaluasi secara case-insensitive untuk mendeteksi kesamaan secara akurat. Pendekatan pencarian komprehensif ini sangat krusial saat mengeksekusi fitur pembatalan tiket. Apabila pencarian ini mengembalikan lebih dari satu entri data (duplikasi nama), sistem akan secara otomatis memicu prosedur mitigasi dengan menampilkan opsi detail kepada pengguna untuk memilih secara pasti transaksi mana yang ingin dibatalkan, sehingga mencegah terjadinya galat operasional atau penghapusan data secara keliru.

3.6.3 Cari Data Database (searchData)

Merupakan utilitas internal untuk mencari nilai kunci (key) dari berkas penyimpanan lokal. Karena data dimuat sebagai struktur Dictionary (Hash Map) di memori, proses pencarian dapat dilakukan secara instan dengan kecepatan konstan ($O(1)$). Sistem

tidak perlu menelusuri data satu per satu, sehingga sangat efisien untuk memvalidasi keberadaan entitas di dalam basis data.

3.6.4 Kalkulasi Nama di Antrean (countNameInQueue)

Berfungsi untuk menghitung jumlah pelanggan dengan nama yang sama di daftar tunggu, fungsi ini digunakan pada fitur cancel antrean, cancel data pemesanan, dan cari data pemesanan guna untuk mendeteksi dan menghindari kesalahan dalam penghapusan atau pencarian data. Menggunakan metode Sequential Search, sistem menelusuri antrean (Queue) dari simpul terdepan (front) hingga terakhir (rear). Setiap kali nama yang dicari cocok, variabel penghitung (counter) akan ditambahkan secara otomatis untuk memastikan validitas keberadaan nama ganda.

3.7 Fitur Sorting

3.7.1 Pengurutan Daftar Kursi (Seat Sorting)

Berfungsi menata ketersediaan kursi secara numerik murni menggunakan algoritma Timsort bawaan Python (sorted()). Sistem mengekstrak digit angka pada penamaan kursi (misal memisahkan "K" dan "10") untuk dikonversi menjadi integer sebagai acuan pengurutan. Metode kustom ini memperbaiki urutan alfabetis standar yang rancu (seperti "K1, K10, K2") menjadi urutan logis matematis ("K1, K2, K10"). Algoritma Timsort diaplikasikan karena sifatnya yang stabil, adaptif, dan berkecepatan tinggi dengan kompleksitas $O(N \log N)$, menjamin data disajikan secara konsisten dan akurat.

3.8 Penyimpanan Data (File Handling)

Aplikasi antrean tiket bioskop ini menggunakan sistem file handling sederhana sebagai basis datanya, yang diimplementasikan di

dalam modul `CRUD/utils/dataOps.py`. Seluruh data operasional disimpan secara permanen di dalam direktori `CRUD/data/`.

3.8.1 Format File yang Digunakan

Aplikasi menggunakan ekstensi file teks biasa (.txt) seperti `data_film.txt`, `log_pemesanan.txt`, `data_antrean.txt`, `temp_seat.txt`, dan `temp_log_pemesanan.txt`. Meskipun berekstensi .txt, format penyimpanan data di dalamnya mengadopsi konsep Comma-Separated Values (CSV). Setiap record data disimpan dalam satu baris (baris per baris), di mana setiap atribut (kolom) dipisahkan oleh tanda koma (.). Untuk atribut yang berisi sekumpulan nilai (list/array) seperti nomor kursi, data tersebut digabungkan menjadi satu string dengan pemisah pipe (|).

3.8.2 Proses Pembacaan File

Proses pembacaan data ditangani oleh fungsi terpusat `getAllData(data_name)` di dalam `dataOps.py`. Aplikasi membuka file teks dengan mode pembacaan teks atau read (`open(file_path, "r", encoding="utf-8")`). Penggunaan context manager (`with`) memastikan file otomatis ditutup setelah selesai dibaca. File dibaca secara iteratif (baris per baris). Setiap baris dibersihkan dari spasi/karakter newline yang berlebih menggunakan metode `.strip()`. Data dalam satu baris kemudian dipecah menggunakan metode `.split(",")` untuk diubah menjadi list. Aplikasi melakukan parsing indeks list tersebut ke dalam struktur data Dictionary bersarang (Nested Dictionary), di mana ID unik (seperti `film_id` atau `log_id`) bertindak sebagai key utama. Fungsi pembacaan ini juga dilengkapi dengan blok `try-except` (khususnya `FileNotFoundError`) yang memungkinkan aplikasi berjalan normal dan mengembalikan Dictionary kosong apabila file yang dituju belum terbuat.

3.8.3 Proses Penyimpanan File

Proses penyimpanan atau pembaharuan data dilakukan melalui fungsi `updateData(data_dict, data_name)`. Aplikasi menerima argumen berupa seluruh data yang telah diperbarui di memori aplikasi (dalam bentuk Dictionary). File target dibuka menggunakan mode `write` (`open(file_path, "w", encoding="utf-8")`). Mode "w" ini akan mengganti seluruh isi file lama dengan isi dari Dictionary yang terbaru (overwrite). Program melakukan perulangan pada Dictionary untuk menyusun kembali setiap atribut ke dalam format aslinya menggunakan `",".join()`. Jika ada data berbentuk list (seperti kumpulan nomor kursi), data tersebut digabungkan dahulu dengan `"|".join()`. Setiap data akhirnya ditulis ke dalam file per baris dan diakhiri dengan karakter penanda baris baru (`\n`).

3.8.4 Tabel Format Penyimpanan dan Contoh

Tabel Data (TXT)	Format Penyimpanan	Contoh
data_film	<i>film_id (String), judul_film (String), kuota_penonton (Integer)</i>	NXR681,the game changer,40
temp_seat	<i>film_id (String), judul_film (String), kursi_1 (String), kursi_2 (String), ...</i>	NXR681,the game changer,K3,K4,K5,K6,K7,K8,K9,K10,K11
data_antrean (COUNTER)	<i>tipe_baris (String), film_id (String), urutan_counter (Integer)</i>	COUNTER,NXR681,2
data_antrean (QUEUE)	<i>tipe_baris (String), film_id (String), nama_pemesan (String), jumlah_tiket (Integer), nomor_kursi (String list dengan pemisah ' '), urutan_antrean (Integer), judul_film (String), tanggal_pesan (String), nama_pemesan_selanjutnya (String / Kosong)</i>	QUEUE,NXR681,Budi,2,K1 K2,1,the game changer,26/06/26 - 11:20:00,Andi [hanya sebagai pemisah visual pada laporan] QUEUE,NXR681,Andi,1,K3,2,the game changer,26/06/26 - 11:22:15,

log_pemesanan & temp_log_pemesanan	<i>log_id (String), nama_pemesan (String), jumlah_tiket (Integer), urutan_antrean (Integer), judul_film (String), tanggal_waktu (String), nomor_kursi (String list dengan pemisah ' ')</i>	IKP452,Budi,2,1,the game changer,26/06/26 - 11:23:05,K1 K2
------------------------------------	--	--

3.9 Validasi Input dan Error Handling

Kesalahan Input	Penanganan
Huruf pada input angka (kuota film, jumlah tiket, atau pilihan nomor duplikat)	Program meminta input ulang dengan menampilkan pesan "Masukkan bilangan yang valid!" atau "Urutan antrean tidak valid!"
Angka atau simbol pada input nama penonton	Program meminta input ulang dengan menampilkan pesan "Nama customer harus berupa huruf dan tidak boleh berupa simbol."
Pemilihan nomor kursi yang tidak valid / sudah terisi	Program meminta input ulang dengan menampilkan peringatan "Kursi tidak valid / sudah terpakai."
Input judul film mengandung tanda koma (,)	Program meminta input ulang ("Judul film tidak boleh mengandung koma (,)") karena tanda koma digunakan sebagai delimiter pada file teks, sehingga dapat merusak format penyimpanan
Menambahkan judul film yang sama / duplikat	Pesan error ditampilkan ("Judul yang sama sudah terdaftar.") dan form pendaftaran diulang kembali
Data tidak ditemukan (melakukan pembatalan pesanan untuk nama yang tidak terdaftar)	Pesan error ditampilkan ("Nama tidak ditemukan.") lalu kembali ke tampilan menu antrean
Memesan tiket melampaui batas wajar atau sisa kursi	Program meminta input ulang dengan menginformasikan batasan yang diizinkan (misal: "Hanya bisa memesan 1-4 tiket!" atau "Hanya tersisa X tiket!")
Melayani antrean saat antrean kosong / tiket sudah habis	Menampilkan pesan peringatan ("Antrean kosong..." atau "Tiket habis...") dan mencegah operasi, lalu mengembalikan user ke menu
Pilihan nomor data di luar rentang (saat menghapus data duplikat)	Program menolak input dan meminta ulang (pesan: "Pilihan harus 0-X!")

Data transaksi kosong saat mencetak laporan	Program menampilkan peringatan ("Tidak ada data transaksi untuk dibuatkan laporan.") dan membatalkan pembuatan file PDF
File database / .txt belum tersedia	Program mencegah terjadinya crash dengan mengabaikan pesan FileNotFoundException dan akan otomatis membuat file .txt baru ketika ada data yang disimpan

BAB IV. PENGUJIAN & ANALISIS

4.1 Skenario Pengujian

Pengujian dilakukan untuk memastikan seluruh struktur data (Queue dan Linked List) serta algoritma (Searching, Sorting, dan File Handling) berjalan dengan semestinya tanpa ada kendala.

No	Skenario Pengujian	Hasil
1	Tambah Data Film (Validasi duplikasi data)	Berhasil
2	Edit Data Film (Update judul dan penyesuaian kuota kursi)	Berhasil
3	Hapus Data Film (Membersihkan antrean, log, dan file terkait)	Berhasil
4	Masuk Antrean (Enqueue) (Pemberian nomor urut otomatis)	Berhasil
5	Layani Antrean (Dequeue) (Pemilihan kursi secara interaktif & pembuatan invoice)	Berhasil
6	Lihat Data (Mencetak isi Queue dan Linked List ke dalam format tabel)	Berhasil
7	Sorting Data (Mengurutkan abjad kursi yang tersedia menggunakan algoritma Sorting)	Berhasil
8	Searching Data (Mencari tiket pesanan di dalam Linked List berdasarkan nama)	Berhasil
9	Batalkan Pesanan (Menghapus Node antrean/tiket dari tengah list dan mengembalikan kuota kursi)	Berhasil
10	Generate Laporan PDF (Menganalisis statistik penjualan dan mengekspor tabel ke .pdf)	Berhasil

4.2 Kelebihan Sistem

Aplikasi Antrean Tiket Bioskop CACB ini memiliki beberapa kelebihan utama yang ditawarkan, antara lain:

1. Data tersimpan permanen secara persisten: Penerapan file handling yang memisahkan log permanen dan sesi temporer memastikan data tidak hilang meski aplikasi ditutup secara tiba-tiba.
2. Program sangat stabil dan tidak mudah crash: Validasi input pada setiap menu dirancang iteratif dengan penanganan error (exception handling) yang tangguh, mencegah berhentinya program akibat salah input dari user.
3. Menu dan navigasi sangat interaktif (User Friendly): Penggunaan library InquirerPy untuk arrow-key selection dan rich untuk pewarnaan serta visualisasi tabel/panel CLI membuat tampilan sangat menarik seperti GUI.
4. Visualisasi Kursi (Seat Mapping): Terdapat fitur penempatan tata letak kursi (denah) secara aktual di layar terminal, memudahkan pengunjung memilih bangku yang kosong (hijau) atau yang sudah terisi (merah).
5. Auto-Generate Laporan Penjualan: Sistem memiliki algoritma analitik yang secara cerdas mendeteksi hari paling ramai, film paling laku, dan mencetak keseluruhan rekam jejak tersebut ke dalam format PDF (reportlab) secara otomatis.
6. Manajemen Duplikasi yang Baik: Aplikasi sudah menerapkan konsep composite key (nama + jumlah + nomor kursi) untuk mencari dan menghapus pesanan, memecahkan masalah jika terdapat customer dengan nama yang persis sama.

4.3 Keterbatasan Sistem

Meskipun sistem telah dirancang dengan struktur algoritma yang kompleks, masih terdapat beberapa keterbatasan yang bisa dikembangkan lebih lanjut di masa depan:

1. Masih berbasis CLI (Command Line Interface): Karena berjalan di lingkungan terminal, daya tarik visual bagi kalangan pengguna awam (customer langsung) mungkin masih kurang jika dibandingkan dengan aplikasi berbasis Web atau Mobile.
2. Belum menggunakan sistem Database Relasional: Sistem masih mengandalkan operasi baca/tulis ke file text murni. Meskipun format mirip CSV ini efektif untuk project skala kecil, ini rentan mengalami kerusakan (corrupt) jika file terhapus atau termodifikasi manual oleh pihak luar.
3. Belum mendukung eksekusi secara Multiuser: Aplikasi ini belum mendukung konkurensi (jaringan server/client). Jika aplikasi ini dijalankan di dua komputer kasir secara bersamaan, berpotensi memicu race condition atau data saling tumpang tindih.
4. Keterbatasan Sesi Penayangan (Showtime): Aplikasi saat ini memproses data antrean untuk satu jadwal penayangan secara global per judul film. Belum terdapat opsi untuk memilih jam tayang (showtime) yang berbeda-beda untuk film yang sama.

BAB V. KENDALA & SOLUSI

Kendala	Solusi
Program crash saat mencetak laporan jika data transaksi kosong (IndexError pada fungsi analyze()).	Menambahkan validasi data (mengecek dictionary kosong) dan menghentikan eksekusi (early return) sebelum proses analitik berjalan.
Fungsi addFilm dan updateFilm menyimpan data tidak valid (integer bernilai negatif / huruf) yang merusak pembacaan data.	Menambahkan validasi tipe data dan jangkauan angka (kuota 1-60) secara ketat di sisi antarmuka (input) maupun fungsi internal API.
Format penyimpanan file CSV rusak (corrupt) jika user memasukkan judul film yang mengandung tanda koma (,).	Menambahkan filter pada input untuk secara tegas menolak pemakaian karakter koma sebelum judul film diproses.
Pembacaan data_antrean.txt gagal (error) dan antrean hilang jika ada baris data yang cacat/rusak (malformed row).	Menambahkan pengecekan panjang elemen kolom (<code>len(list_row) < 7</code>) lalu melewati (skip) baris yang bermasalah tersebut.
Nomor urut antrean melompat (terdapat gap) jika ada pengunjung di posisi depan yang pesannya dibatalkan.	Menambahkan logika pengurangan urutan (<code>self.urutan -= 1</code>) pada class Queue saat method <code>cancelQueue()</code> membatalkan pengunjung.
Fungsi pembatalan antrean (<code>cancelQueue</code>) mengalami crash (AttributeError) jika dieksekusi pada antrean yang sama sekali kosong.	Menambahkan percabangan validasi (<code>if q.isEmpty()</code>) untuk menolak eksekusi eksternal terhadap memori antrean yang kosong.
Judul film yang ditambahkan ke dalam antrean (<code>enqueue</code>) tersimpan sebagai teks "None" yang dapat memicu ketidaksesuaian data pencarian.	Menyematkan parameter <code>judul_film</code> secara langsung saat instansiasi/pembuatan Node antrean baru (bukan sekadar parameter opsional).
Proses "Batalkan Antrean" tidak memulihkan (refund) ketersediaan sisa tiket dan nomor kursi kembali ke sistem.	Mendorong (<code>append</code>) kembali nomor kursi yang batal ke variabel <code>available_seat</code> serta menjumlahkan kembali kuota <code>available_ticket</code> .

Pengunjung yang antre terjebak (tanpa notifikasi) saat sisa tiket telah habis di-pesan oleh pengunjung yang tepat berada di depannya.	Memberikan pesan notifikasi "Tiket habis" dan secara sistematis menolak melanjutkan form layanan jika sisa tiket divalidasi 0.
Tampilan UI denah tata letak kursi (layar bioskop) tiba-tiba menghilang/kosong saat tidak ada lagi sisa tiket yang tersedia.	Memperbaiki acuan fungsi <code>display_seats</code> agar mencetak antarmuka berdasarkan kuota master film, bukan dari panjang list sisa tiket.
Pencetakan invoice dengan waktu eksekusi sangat berdekatan (di detik yang sama) membuat file PDF saling menimpa (collision).	Menyempurnakan format penamaan file invoice (PDF) dengan menambahkan kode unik seperti UUID atau presisi hingga milidetik.
Terlalu banyak tab browser web yang terbuka menumpuk akibat perintah <code>auto-open</code> saat mencetak invoice transaksi.	Menghapus atau menonaktifkan kode pemanggilan library <code>webbrowser.open()</code> pada fungsi pembuat invoice.
Kuota kursi yang di-update menjadi lebih besar tidak bertambah/melebar di sesi antrean pemilihan kursi (<code>temp_seat.txt</code>).	Menambahkan logika propagasi (sinkronisasi rekonsiliasi) di file temporer agar jumlah kursi otomatis mengikuti kuota master terbaru.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

6.1 Fateeh Falaah Hendharto (J0403251098)

Role: Sr. Engineer, Project Advisor, dan QA

Fitur utama yang dikerjakan:

1. Manajemen data pemesanan.
2. Manajemen pemesanan kursi.
3. Manajemen pemesanan tiket.

Tugas lainnya yang dikerjakan:

1. Menjaga kualitas kode dan program tetap baik agar efektif dan efisien saat dijalankan.
2. Melakukan refactoring sebagai upaya menjaga kualitas kode dan program.
3. Debugging cross-task (lintas tanggung jawab fitur).
4. Membantu melakukan testing dan QA pada saat development version 2 serta menjaga kualitas kode pada development version 2.

6.2 Muhammad Lutfi Ramadhan Warendra (J0403251040)

Role: Engineer & Project Advisor

Fitur utama yang dikerjakan:

1. Menu awal.
2. Manajemen data film.
3. Invoice/struk pemesanan.

Tugas lainnya yang dikerjakan:

1. Membantu mengerjakan fitur laporan penjualan tiket pada saat development version 2.

2. Debugging.

6.3 Wildhan Dzikrihantara (J0403251098)

Role: Project Manager & Lead Engineer

Fitur utama yang dikerjakan:

1. Menu pilih film.
2. Menu Sistem Antrean
3. Manajemen Antrean.

Tugas lainnya yang dikerjakan:

1. Menentukan tujuan, fitur-fitur, dan fungsi/algoritma apa yang digunakan pada aplikasi serta menentukan batch development (sprint & versions).
2. Merancang dan menentukan flow, arsitektur, dan interface aplikasi.
3. Melakukan penambahan fitur besar dan mengeksekusi development pada version 2: session (state), perubahan besar interface, dan laporan penjualan tiket untuk Sistem Informasi Antrean Tiket.
4. Debugging cross-task (lintas tanggung jawab fitur).

Sumber:

1. https://github.com/wilanta/ASD_B_4/graphs/contributors?all=1
2. https://github.com/wilanta/ASD_B_4/graphs/commit-activity

BAB VII.

KESIMPULAN & SARAN

7.1 Kesimpulan

Proyek Bioskop CACB berhasil menghasilkan aplikasi manajemen antrean tiket bioskop berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python yang fungsional, stabil, dan interaktif. Sistem ini berhasil mengimplementasikan struktur data Queue dengan prinsip FIFO untuk pengelolaan antrean pelanggan, serta Linked List untuk pencatatan riwayat transaksi secara dinamis, sehingga seluruh alur pelayanan kasir dapat berjalan secara terstruktur dan efisien.

Selain itu, proyek ini berhasil menerapkan mekanisme file handling berbasis file TXT berformat CSV sebagai basis data lokal yang menjamin kegigihan data antar sesi, dilengkapi dengan validasi input yang tangguh dan penanganan error yang komprehensif sehingga program tidak mudah crash akibat kesalahan pengguna. Fitur-fitur tambahan seperti pencarian tiket menggunakan Sequential Search, pengurutan nomor kursi menggunakan Timsort, visualisasi denah kursi di terminal, serta ekspor invoice dan laporan penjualan ke format PDF turut memperkaya nilai fungsional dari sistem yang dibangun.

7.2 Saran

Untuk pengembangan ke depan, sistem ini disarankan untuk bertransisi dari antarmuka CLI menuju antarmuka berbasis web atau mobile agar lebih mudah diakses dan lebih menarik bagi pengguna awam. Selain itu, mekanisme penyimpanan data sebaiknya dimigrasikan dari file teks ke sistem basis data relasional seperti PostgreSQL atau SQLite guna meningkatkan keamanan, integritas, dan skalabilitas data agar terhindar dari risiko kerusakan file akibat modifikasi eksternal. Sistem juga perlu

dikembangkan untuk mendukung eksekusi secara multiuser dengan arsitektur server-client, serta dilengkapi dengan fitur manajemen jadwal tayang (showtime) yang memungkinkan satu judul film memiliki beberapa sesi penayangan berbeda dalam satu hari operasional.

DAFTAR PUSTAKA

1. **Cormen et al.** Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). Cambridge, MA: The MIT Press.
2. **Goodrich et al.** Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6th ed.). Hoboken, NJ: John Wiley & Sons.

LAMPIRAN

A. Link Github

https://github.com/wilanta/ASD_B_4

B. Screenshoot Program

```
BIOSKOP CACE

-----
Pilih menu yang ingin diakses
Pilih untuk melanjutkan.
-----

? Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih
> 1. Sistem Informasi Antrean
  2. Daftar Film
  3. Tambah Film
  4. Laporan Penjualan Tiket
  0. Keluar
```

```
Pilih Film
Pilih untuk melanjutkan.

? Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih
> the game changer [40 kuota]
  Example Title [40 kuota]
  --- Keluar ---
```

```
Pilih Film
Pilih untuk melanjutkan.

? the game changer [40 kuota]
? Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih
> 1. Update
  2. Delete
  0. Kembali
```

```
Pilih Film
> Pilih untuk melanjutkan.

? the game changer [40 kuota]
? 1. Update

Ubah Film
_____

Judul Film      : the game changer
Kuota Penonton  : 40

Kosongkan isian jika tidak ingin mengganti data

Judul Film Baru : 
Kuota Baru      : 
```

```
----- BIOSKOP CACB -----

Judul Film      : the game changer
Tiket Tersedia : 38

----- Sistem Informasi Antrean -----

Pilih opsi
? Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih █
> 1. Masuk Antrean
  2. Layani Antrean
  3. Lihat Antrean
  4. Lihat Data Pemesanan
  5. Batalkan Antrean
  6. Batalkan Pemesanan
  7. Cari Data Pemesanan
  8. Reset Antrean dan Pemesanan
  0. Kembali
```

```
----- BIOSKOP CACB -----

Judul Film      : the game changer
Tiket Tersedia : 38

----- Sistem Informasi Antrean -----

Pilih opsi
? 1. Masuk Antrean

Masukkan nama penonton [Kosongkan isi dan enter untuk kembali]: Wil
```

Daftar Antrean

No	Nama
1	dhan
2	wil

[Tekan Enter untuk Kembali]

Daftar Pemesanan Tiket

No	Nama	Jumlah Tiket	Nomor Kursi	Tanggal
1	wil	2	K1 K2	26/06/26 - 16:41:38

[Tekan Enter untuk Kembali]

BIOSKOP CACB

Judul Film : the game changer
Tiket Tersedia : 38

Sistem Informasi Antrean

Pilih opsi
? 2. Layani Antrean

Masukkan jumlah tiket yang dipesan [Kosongkan isi dan enter untuk kembali]: 2

Pilih Kursi Tiket ke-1

LAYAR BIOSKOP

X	X	K3	K4	K5	K6	K7	K8	K9	K10
K11	K12	K13	K14	K15	K16	K17	K18	K19	K20
K21	K22	K23	K24	K25	K26	K27	K28	K29	K30
K31	K32	K33	K34	K35	K36	K37	K38	K39	K40

Masukkan kursi tiket ke-1 (contoh: K12):

Nama customer yang akan dibatalkan (Enter untuk kembali):

Nama customer yang akan dicari (Enter untuk kembali):

BIOSKOP CACB

Jalan Lodaya II, Bogor

Judul : the game changer

Nama : wil

Kursi : K6

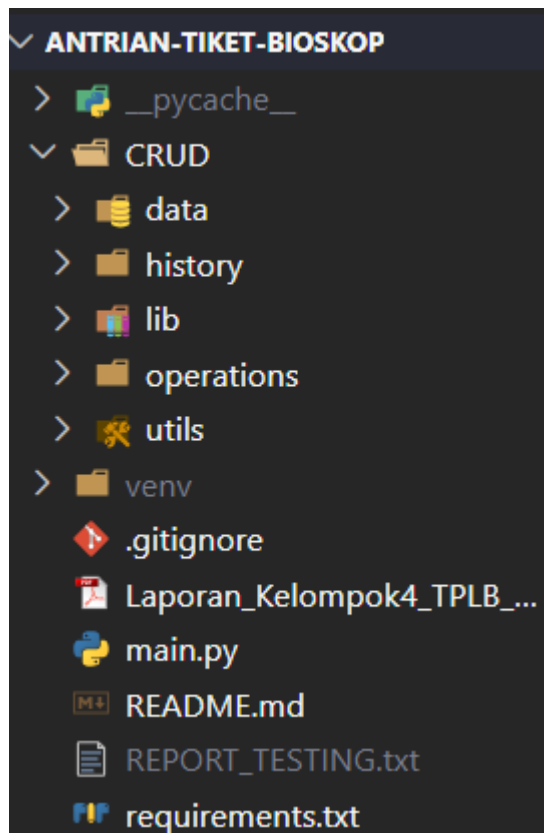
Tanggal: 26/06/2026 16:45:44

Dilayani : Admin

Terima Kasih

Silakan Datang Kembali

C. Struktur Proyek (Folder)



D. Commit History Github

https://github.com/wilanta/ASD_B_4/commits/master/

E. Source Code Utama (main.py & antrean.py)

=====

====

Main Menu

Titik masuk utama aplikasi BIOSKOP CACB.

Berisi navigasi menu dan kontrol ke seluruh fitur sistem.

Kontributor : Wildhan Dzikrihantara, M. Lutfi Ramadhan Warendra

Fungsi/fitur:

1. main - Menampilkan menu utama dan mengarahkan ke fitur yang dipilih

```

=====
=====
"""

# Import features yang akan digunakan di main

# Antrean
from CRUD.operations.antrean import sistemAntrean

# film
from CRUD.operations.film import pilihFilm, addFilm, updateFilm,
deleteFilm

from CRUD.utils.dataOps import getAllData

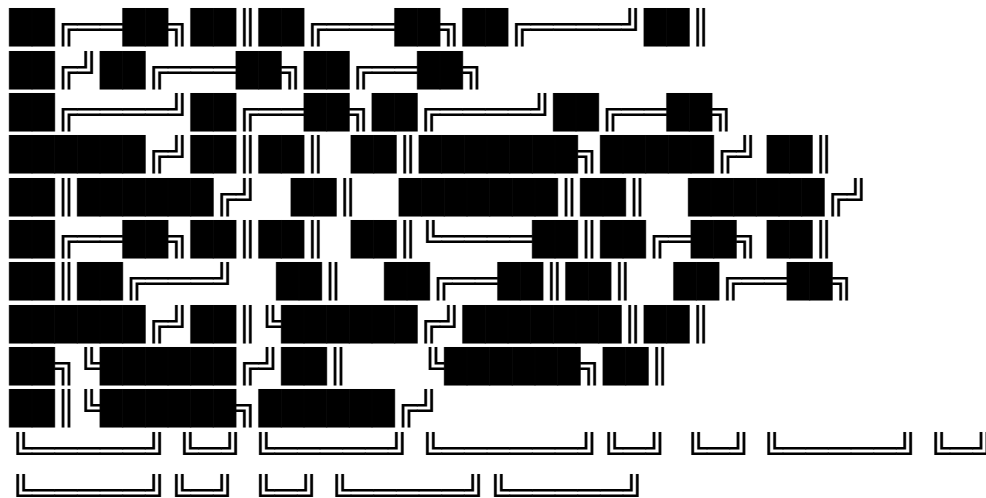
# Report
from CRUD.operations.report import generateReport

# Untuk interface
from InquirerPy import inquirer
from rich import print
from CRUD.utils.clear import _clear
from CRUD.utils.loading import _processing
import shutil
from rich.console import Console

console = Console()

# -----
# Nama fungsi: main
# Penjelasan fungsi : Untuk tampilan dan kontrol main menu.
# -----
def main():
    while True:
        _clear()
        print("""
[bold white]
██████████ ███ ██████████ ██████████ ██████████ ██████████ ██████████
██████████ ███ ██████████ ██████████ ██████████ ██████████ ██████████

```



```
[/bold white]
""")
    width = shutil.get_terminal_size().columns
    # Cetak garis pemisah sepanjang lebar terminal
    print("-" * width)

    print("[bold white]Pilih menu yang ingin diakses[/bold white]")
    print("[dim]Pilih untuk melanjutkan.[/dim]\n")

    # Cetak garis pemisah sebelum daftar pilihan
    print("-" * width)

    # Menampilkan pilihan dan meminta pilihan dari user
    choice = inquirer.select(
        message="",
        choices=[
            "1. Sistem Informasi Antrean",
            "2. Daftar Film",
            "3. Tambah Film",
            "4. Laporan Penjualan Tiket",
            "0. Keluar",
        ],
        default="Auto (match terminal)",
        pointer=">",
        instruction="Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih",
    ).execute()
```

```

# Mengecek nilai dari variabel 'pilihan'
match choice:
    case "1. Sistem Informasi Antrean": # Sistem Antrean
        # Memanggil pilihFilm untuk memilih film yang akan dioperasikan
        antreannya
        film_id = pilihFilm()

        # Validasi film_id dan panggil sistem antrean
        if film_id:
            _clear()
            sistemAntrean(film_id)
        else:
            _clear()
            print("Tidak sesuai nomor di urutan. Kembali ke menu utama...")

    case "2. Daftar Film": # Show film
        _clear()
        film_id = pilihFilm()

        # Film_id validator
        if film_id is None:
            continue

        # Tampilkan sub-menu untuk update atau delete film
        choice = inquirer.select(
            message="",
            choices=[
                "1. Update",
                "2. Delete",
                "0. Kembali",
            ],
            default="Auto (match terminal)",
            pointer=">",
            instruction="Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk
memilih",
        ).execute()

        match choice:
            case "1. Update": # Update Film
                film = getAllData("data_film").get(film_id)

```



```

# Tampilkan header form ubah film
console.print("\n[bold]Ubah Film[/bold]")

console.print("[dim]-----[/dim]\n")

# Tampilkan data film saat ini
console.print(
    f"[cyan]Judul Film[/cyan]      : {film['judul_film']}"
)
console.print(
    f"[cyan]Kuota Penonton[/cyan] : {film['kuota_penonton']}"
)

console.print(
    "\n[dim]Kosongkan isian jika tidak ingin mengganti
data[/dim]\n"
)

# Input judul dan kuota baru dengan validasi
while True:
    judul = console.input(
        "[cyan]Judul Film Baru[/cyan] : "
    ).strip()

    kuota_penonton = console.input(
        "[cyan]Kuota Baru[/cyan] : "
    ).strip()

    # Judul tidak boleh mengandung koma (delimiter CSV)
    if "," in judul:
        console.print(
            "\n[red]✗[/red] Judul film tidak boleh mengandung
koma (,)\n"
        )
        continue

    # Jika terdapat data film yang sama di database dengan yang
    ditambahkan, hentikan penambahan film

```

```

data_film = getAllData("data_film")
if any(
    judul == film["judul_film"]
    for film in data_film.values()
):
    console.print(
        "[red] ✗ Tidak dapat mengganti judul film[/red]: judul
yang sama sudah terdaftar."
    )
    continue

# Kuota boleh kosong atau angka 1-60
if not kuota_penonton or (
    kuota_penonton.isdigit()
    and 0 < int(kuota_penonton) <= 60
):
    break

console.print(
    "\n[red] ✗ [/red] Kuota penonton harus berupa angka valid
dan maksimal 60!\n"
)

_processing("Menyimpan perubahan...")

# Panggil updateFilm dengan nilai baru (atau kosong = keep)
updateFilm(
    judul_film=judul,
    kuota_penonton=kuota_penonton,
    film_id=film_id,
)

_clear()

console.print("\n[green] ✓ [/green] Film berhasil diubah!\n")

case "2. Delete": # Delete Film
_processing("Menghapus film...")
# Panggil deleteFilm untuk hapus dari database
deleteFilm(film_id)

```

```

        _clear()
        print("Film berhasil dihapus!")

    case "0. Kembali": # Kembali ke menu utama
        _clear()
        print("Kembali ke menu utama.")
        continue

    case _: # Pilihan tidak valid
        print("Pilihan tidak valid!")
        continue

case "3. Tambah Film": # Tambah Film
    _clear()

    # Flag untuk menandai form kosong (batal tambah)
    empty = True

    # Tampilkan header form tambah film
    console.print("\n[bold]Tambah Film[/bold]")

    console.print("[dim]_____[/dim]")
    console.print(
        "[dim]Kosongkan isian untuk membatalkan penambahan"
        "film[/dim]\n"
    )

    # Input judul & kuota dengan validasi
    while True:
        judul = console.input("[cyan]Judul Film[/cyan] \t: ").strip()
        kuota_penonton = console.input(
            "[cyan]Kuota Penonton[/cyan] \t: "
        ).strip()

        # Jika salah satunya kosong, anggap user membatalkan
        if not judul or not kuota_penonton:
            break

```

```

# Judul tidak boleh mengandung koma (delimiter CSV)
if "," in judul:
    console.print(
        "\n[red] ✗ [/red] Judul film tidak boleh mengandung koma
(,)\n"
    )
    continue

# Validasi kuota sebagai angka 1-60
if kuota_penonton.isdigit() and 0 < int(kuota_penonton) <= 60:
    empty = False
    break

console.print(
    "\n[red] ✗ [/red] Kuota penonton harus berupa angka valid dan
maksimal 60 orang!\n"
)

# Jika form kosong, batalkan penambahan
if empty:
    console.print("\n[yellow]![/yellow] Membatalkan penambahan
film...")
    continue

_processing("Menyimpan film...")

# Tambah film ke database
add_status = addFilm(judul, int(kuota_penonton))

if not add_status:
    console.print(
        "[red] ✗ Gagal menambahkan film[/red]: judul yang sama
sudah terdaftar."
    )
    input("[Tekan enter untuk kembali]")

_clear()

console.print("\n[green] ✓ [/green] Film berhasil ditambahkan!\n")

```

```

case "4. Laporan Penjualan Tiket":
    _clear()
    # Ambil data log_pemesanan permanent sebagai sumber laporan
    data = getAllData("log_pemesanan")

    # Jika belum ada data, tampilkan pesan dan kembali ke menu
    if not data:
        print("Data log_pemesanan kosong!")
        continue

    _processing("Membuat laporan...")
    # Generate laporan penjualan tiket
    generateReport(data)

    _clear()
    print("Laporan berhasil dibuat!")

case "0. Keluar": # Kembali ke menu utama
    _clear()
    print("Program dihentikan.")
    break

case _: # Pilihan tidak valid
    print("Pilihan tidak valid!\n")

if __name__ == "__main__":
    main()

```

"""

=====

====

Manajemen Sistem Antrean Pesanan Tiket Film.

Fungsi/fitur:

1. Masuk Antrean
2. Layani Antrean
3. Lihat Antrean
4. Lihat Data Pemesanan
5. Batalkan Antrean
6. Batalkan Pemesanan
7. Cari Data Pemesanan
8. Reset Antrean dan Pemesanan

=====

====

"""

queue

from CRUD.operations.queue import Queue

linked list / ticket

from CRUD.operations.ticket import Ticket

order

from CRUD.operations.order import orderKursi, resetOrder

invoice

from CRUD.operations.invoice import invoice

utilities & libraries

from CRUD.utils.idGenerator import generateID

from CRUD.utils.dataOps import getAllData, searchData, updateData

from CRUD.utils.seatSort import seat_sort

from CRUD.utils.node import Node

from CRUD.utils.clear import _clear

from CRUD.utils.loading import _processing

from CRUD.utils.deleteTempSessions import (
 deleteTempPemesanan,
 deleteTempSeat,

```

        deleteLogPemesananByComposite,
        searchTempLogPemesananDuplicates,
    )

# Untuk interface
from rich.console import Console
from rich.panel import Panel
from InquirerPy import inquirer
from rich import print

# -----
# Nama fungsi: _load_queue
# Penjelasan fungsi : Memuat state antrean dari file data_antrean.txt
# berdasarkan film_id yang diberikan.
# -----
def _load_queue(film_id: str, judul_film: str) -> Queue:
    """
    Memuat state antrean dari file untuk film tertentu.

    Args:
        film_id (str): ID film.
        judul_film (str): Judul film.

    Returns:
        Queue: Objek queue yang sudah dimuat dengan data dari file.
    """
    # Inisialisasi queue kosong
    q = Queue()
    data = getAllData("data_antrean")

    # Memuat nodes dari file jika sudah ada state tersimpan
    if film_id in data:
        # Ambil counter urutan dan list node dari data film
        q.urutan = data[film_id].get("urutan_counter", 1)
        nodes = data[film_id].get("nodes", [])

        # Merekonstruksi linked list secara berurutan
        prev = None
        for node_dict in nodes:

```

```

# Rekonstruksi node dari dictionary dengan fallback value
node = Node(
    nama=node_dict["nama"],
    jumlah_tiket=node_dict.get("jumlah_tiket"),
    nomor_kursi=node_dict.get("nomor_kursi"),
    urutan_antrean=node_dict.get("urutan_antrean"),
    judul_film=node_dict.get("judul_film", judul_film),
)
node.create_at = node_dict.get("create_at", node.create_at)

# Set front untuk node pertama, sambungkan node setelahnya
if prev is None:
    q.front = node
else:
    prev.next = node

prev = node

# Set rear ke node terakhir hasil rekonstruksi
q.rear = prev if prev else None

return q

# -----
# Nama fungsi: _save_queue
# Penjelasan fungsi : Menyimpan state antrean saat ini ke file
# data_antrean.txt sesuai format dataOps.py.
# -----
def _save_queue(film_id: str, q: Queue):
    """
    Menyimpan state antrean saat ini ke file data_antrean.txt.

    Args:
        film_id (str): ID film.
        q (Queue): Objek queue yang state-nya akan disimpan.
    """

# Mengumpulkan semua node menjadi list dictionary
nodes_list = []

```



```

current = q.front

# Traversal queue dan konversi setiap node ke dict
while current is not None:
    nodes_list.append(
        {
            "nama": current.nama,
            "jumlah_tiket": current.jumlah_tiket,
            "nomor_kursi": current.nomor_kursi,
            "urutan_antrean": current.urutan_antrean,
            "judul_film": current.judul_film,
            "create_at": current.create_at,
            "next": current.next.nama if current.next else None,
        }
    )

    current = current.next

# Membaca data lama, menambahkan/memperbarui entry film ini, lalu
menyimpan
data = getAllData("data_antrean")

data[film_id] = {
    "urutan_counter": q.urutan,
    "nodes": nodes_list,
}

updateData(data, "data_antrean")

# -----
# Nama fungsi: _load_tickets
# Penjelasan fungsi : Memuat data pemesanan dari temp_log_pemesanan.txt
# ke linked list Ticket untuk film tertentu.
# -----
def _load_tickets(judul_film: str, ll: Ticket):
    """
    Memuat record pemesanan dari file ke linked list Ticket.

```

Args:

judul_film (str): Judul film sebagai kunci penyaringan record.

ll (Ticket): Objek Ticket linked list sebagai target muat data.

"""

```
log = getAllData("temp_log_pemesanan")
```

```
# Iterasi semua record dan filter yang sesuai judul_film
```

```
for cust_id, record in log.items():
```

```
    if record.get("judul") == judul_film:
```

```
        kursi_raw = record.get("nomor_kursi")
```

```
        # Normalisasi format nomor_kursi (bisa string dengan | atau list)
```

```
        if isinstance(kursi_raw, str):
```

```
            nomor_kursi = [k.strip() for k in kursi_raw.split("|") if k.strip()]
```

```
        elif isinstance(kursi_raw, list):
```

```
            nomor_kursi = kursi_raw
```

```
        else:
```

```
            nomor_kursi = []
```

```
# Tambahkan ke linked list Ticket
```

```
ll.addTicket(
```

```
    nama=record.get("nama", ""),
```

```
    jumlah_tiket=int(record.get("jumlah_tiket", 0)),
```

```
    nomor_kursi=nomor_kursi,
```

```
    urutan_antrean=int(record.get("urutan_antrean", 0)),
```

```
    judul_film=record.get("judul", judul_film),
```

```
    customer_id=cust_id,
```

```
)
```

```
# -----
```

```
# Nama fungsi: _save_tickets
```

```
# Penjelasan fungsi : Menyimpan state Ticket linked list saat ini
```

```
# ke temp_log_pemesanan.txt (hanya records film ini).
```

```
# -----
```

```
def _save_tickets(judul_film: str, ll: Ticket):
```

```
    """
```

Menyimpan state terbaru linked list Ticket
ke temp_log_pemesanan tanpa menduplikasi data lama.

Args:

 judul_film (str): Judul film.

 ll (Ticket): Objek linked list Ticket.

"""

Mengambil semua data lama

temp_log = getAllData("temp_log_pemesanan")

Menghapus semua data film yang sama agar tidak duplikat saat re-save

temp_log = {

 log_id: data

 for log_id, data in temp_log.items()

 if data.get("judul") != judul_film

}

Traversal linked list

current = ll.head

while current is not None:

 # Menambahkan state terbaru ke temp_log

 temp_log[current.customer_id] = {

 "nama": current.nama,

 "jumlah_tiket": str(current.jumlah_tiket),

 "urutan_antrean": str(current.urutan_antrean),

 "judul": current.judul_film,

 "nomor_kursi": current.nomor_kursi,

 "date": str(current.create_at),

 }

 current = current.next

Simpan kembali ke file/database

updateData(temp_log, "temp_log_pemesanan")

Nama fungsi: _reconcile_seat_availability

Penjelasan fungsi : Merekonsiliasi available_seat dengan data pemesanan
 # yang ada di linked list Ticket untuk memastikan konsistensi.

```
def _reconcile_seat_availability(available_seat: list, ll: Ticket) -> list:
    """
```

Merekonsiliasi daftar kursi tersedia dengan data pemesanan aktif.
 Menghapus kursi dari available_seat jika sudah dipesan di linked list.

Args:

available_seat (list): Daftar kursi yang tersedia.

ll (Ticket): Linked list berisi data pemesanan aktif.

Returns:

list: Daftar kursi tersedia yang sudah direkonsiliasi.

"""

Kumpulkan semua kursi yang sudah dipesan dari linked list

booked_seats = set()

current = ll.head

Iterasi seluruh tiket dan kumpulkan kursi yang sudah dibooking

while current is not None:

if current.nomor_kursi:

for seat in current.nomor_kursi:

if seat and seat.strip():

booked_seats.add(seat.strip())

current = current.next

Hapus kursi yang sudah dipesan dari available_seat

reconciled_seats = [seat for seat in available_seat if seat not in
 booked_seats]

return reconciled_seats

Nama fungsi: sistemAntrean

Penjelasan fungsi : Untuk memanage sistem antrian suatu film bioskop.

```
def sistemAntrean(film_id: str):
```

"""Menu Sistem Antrean"""

```

film_dict = searchData(film_id, "data_film")

# Load available seat
temp_seat_data = searchData(film_id, "temp_seat")

available_ticket: int
available_seat: list

current_kuota = int(film_dict[film_id]["kuota_penonton"])

if temp_seat_data:
    # available seat loaded from database
    available_seat = list(temp_seat_data[film_id]["available_seat"])

    # Rekonsiliasi dengan kuota terbaru agar perubahan kuota film
    # (melalui menu Daftar Film -> Update) terpropagasi ke temp_seat.
    # - Jika kuota naik: tambahkan kursi baru (K_max+1 .. K_new).
    # - Jika kuota turun: kursi dengan nomor di atas kuota baru
    #   dihapus dari daftar tersedia (kursi yang sudah terjual
    #   tetap aman karena sudah tidak ada di available_seat).
    existing_numbers = sorted(
        int(s[1:])
        for s in available_seat
        if isinstance(s, str) and s.startswith("K") and s[1:].isdigit()
    )
    max_existing = existing_numbers[-1] if existing_numbers else 0

    if max_existing < current_kuota:
        # Tambah kursi baru jika kuota dinaikkan
        available_seat.extend(
            "K" + str(i) for i in range(max_existing + 1, current_kuota + 1)
        )
    elif max_existing > current_kuota:
        # Hapus kursi dengan nomor di atas kuota baru
        available_seat = [s for s in available_seat if int(s[1:]) <=
current_kuota]

    available_seat = seat_sort(available_seat)

```

```

else:
    # create new available seat list and write to database
    available_seat = ["K" + str(i) for i in range(1, current_kuota + 1)]

    judul_film = film_dict[film_id]["judul_film"]

    # Load queue from file if exists
    q = _load_queue(film_id, judul_film)

    # Load booking/ticket data
    ll = Ticket()
    _load_tickets(judul_film, ll)

    # Rekonsiliasi available_seat dengan data pemesanan aktif
    # untuk memastikan kursi yang sudah dipesan tidak bisa dipilih lagi
    # (mencegah inkonsistensi data antar sesi)
    available_seat = _reconcile_seat_availability(available_seat, ll)
    available_ticket = len(available_seat)

    # Tulis hasil rekonsiliasi ke database agar persisten antar sesi
    temp_seat = getAllData("temp_seat")
    temp_seat[film_id] = {
        "judul_film": judul_film,
        "available_seat": available_seat,
    }
    updateData(data_dict=temp_seat, data_name="temp_seat")

    # Menu utama
    while True:
        console = Console()

        tiket = available_ticket

        content = f"""
        [bold white]Judul Film    :[/bold white] {judul_film}
        [bold white]Tiket Tersedia :[/bold white] {tiket}
        """

        _clear()

```

```

print("\n\n\n")

console.print(
    Panel(
        content,
        title="[bold yellow]BIOSKOP CACB[/bold yellow]",
        subtitle="[dim]Sistem Informasi Antrean[/dim]",
        border_style="yellow",
        padding=(1, 2),
    )
)

print("\n[bold white]Pilih opsi[/bold white]")

choice = inquirer.select(
    message="",
    choices=[
        "1. Masuk Antrean",
        "2. Layani Antrean",
        "3. Lihat Antrean",
        "4. Lihat Data Pemesanan",
        "5. Batalkan Antrean",
        "6. Batalkan Pemesanan",
        "7. Cari Data Pemesanan",
        "8. Reset Antrean dan Pemesanan",
        "0. Kembali",
    ],
    default="Auto (match terminal)",
    pointer=">",
    instruction="Gunakan ↑ ↓ untuk memindahkan opsi, Enter untuk memilih",
).execute()

match choice:
    case "1. Masuk Antrean":
        if available_ticket > 0:
            empty = True

    while True:
        # Input nama penonton dengan validasi huruf

```

```

nama_penonton = input(
    "\nMasukkan nama penonton [Kosongkan isi dan enter untuk
kembali]: "
).strip()

if not nama_penonton:
    break

if nama_penonton.strip().replace(" ", "").isalpha():
    empty = not empty
    break

print(
    "[bold red>Nama penonton harus berupa huruf dan tidak
boleh berupa simbol.[/bold red]"
)

if empty:
    _clear()
    print("[bold red]Antrean dibatalkan.[/bold red]")
    continue

# Tambahkan customer ke antrean dan simpan state
q.enqueue(nama_penonton)

_save_queue(film_id, q)

_processing("Memproses...")
_clear()

print(f"[green]{nama_penonton} masuk antrean.[/green]")

else:
    print("[bold red]Antrean Penuh, tiket habis![/bold red]")

case "2. Layani Antrean":
    # Validasi: antrean tidak boleh kosong
    if q.isEmpty():
        _clear()

```



```

        print(
            "[bold red]Antrean kosong, tidak ada yang bisa dilayani![bold
red]"
        )

        input("\nTekan Enter untuk kembali...")
        continue

# Validasi: harus ada tiket tersedia
if available_ticket < 1:
    _clear()

    print(
        "[bold red]Tiket habis, tidak ada yang bisa dilayani![bold red]"
    )

    input("\nTekan Enter untuk kembali...")
    continue

# Maksimum tiket yang boleh dipesan per customer
max_kursi_per_cust = 4

out = False
while True:
    try:
        # Input jumlah tiket dengan validasi range
        user_input = input(
            "\nMasukkan jumlah tiket yang dipesan [Kosongkan isi dan
enter untuk kembali]: "
        ).strip()

        if not user_input:
            out = True
            break

        user_ticket = int(user_input)

        if user_ticket < 1 or user_ticket > max_kursi_per_cust:
            print("[bold red]Hanya bisa memesan 1-4 tiket![bold red]")
            continue

```

```

        if user_ticket > available_ticket:
            print(
                f"[bold red]Hanya tersisa {available_ticket} tiket![/bold
red]"
            )
            continue

        break

    except ValueError:
        print("[bold red]Masukkan bilangan yang valid![/bold red]")

    if out:
        continue

    # Proses pemilihan kursi interaktif
    ticket_amount, selected_seat, available_seat = orderKursi(
        user_ticket, available_seat, current_kuota
    )

    # Kurangi tiket tersedia
    available_ticket -= ticket_amount

    # remove selected_seat from database and write to database
    temp_seat = getAllData("temp_seat")

    temp_seat[film_id] = {
        "judul_film": film_dict[film_id]["judul_film"],
        "available_seat": available_seat,
    }

    updateData(data_dict=temp_seat, data_name="temp_seat")

    # Update data customer di front queue dengan tiket
    q.updateQueue(
        jumlah_tiket=ticket_amount,
        nomor_kursi=selected_seat,
        judul_film=judul_film,
    )

```

```

# Generate ID log baru untuk entri permanent
log_id = generateID()

# Ambil info customer yang sedang dilayani
nama_customer = q.front.nama
urutan_antrean = q.front.urutan_antrean

# Tambahkan record ke linked list ticket
ll.addTicket(
    nama=nama_customer,
    jumlah_tiket=ticket_amount,
    nomor_kursi=selected_seat,
    urutan_antrean=urutan_antrean,
    judul_film=judul_film,
    customer_id=log_id,
)

# save permanent log
log_pemesanan = getAllData("log_pemesanan")

log_pemesanan[log_id] = {
    "nama": nama_customer,
    "jumlah_tiket": ticket_amount,
    "urutan_antrean": urutan_antrean,
    "judul": judul_film,
    "date": q.front.create_at,
    "nomor_kursi": selected_seat,
}

# Simpan log permanent ke database
updateData(data_dict=log_pemesanan,
data_name="log_pemesanan")

_processing("Memproses...")
_clear()

# Cetak invoice untuk customer
invoice(judul=judul_film, nama=nama_customer,
kursi=selected_seat)

```

```

# Hapus customer dari front queue (sudah dilayani)
served_node = q.dequeue()

# Simpan state queue dan ticket terbaru
_save_queue(film_id, q)
_save_tickets(judul_film, ll)

print(
    f"[cyan]Invoice berhasil dicetak | {served_node.nama} telah
dilayani![/cyan]\n"
)

case "3. Lihat Antrean":
    _clear()

    q.showQueue()

    input("[Tekan Enter untuk Kembali]")

case "4. Lihat Data Pemesanan":
    _clear()

    ll.showTickets()

    input("[Tekan Enter untuk Kembali]")

case "5. Batalkan Antrean":
    _clear()

    if q.isEmpty():
        print(
            "[bold red]Antrean kosong, tidak ada data yang bisa
dibatalkan![/bold red]"
        )

        input("\nTekan Enter untuk kembali...")
        continue

while True:

```

```

nama = input(
    "Nama customer yang akan dibatalkan (Enter untuk kembali): "
).strip()

if nama.strip().replace(" ", "").isalpha() or not nama:
    break

print(
    "[bold red]Nama customer harus berupa huruf dan tidak boleh  

berupa simbol.[/bold red]"
)

if not nama:
    _clear()
    print("Kembali ke menu...")
    continue

# Hapus customer dari antrean
canceled_user, canceled_urutan = q.cancelQueue(nama)

if not canceled_user:
    print("[bold red>Nama tidak ditemukan.[/bold red]")

    input("\nTekan Enter untuk kembali...")
    continue

_processing("Membatalkan antrean...")

# Simpan state queue dan ticket setelah pembatalan
_save_queue(film_id, q)
_save_tickets(judul_film, ll)

_clear()

# Tampilkan pesan sukses dengan informasi urutan (jika ada)
print(
    f"[green]Antrean atas nama {canceled_user} {f' dengan urutan  

{canceled_urutan}' if canceled_urutan else ''} berhasil dibatalkan.[/green]"
)

```

```

case "6. Batalkan Pemesanan":
    _clear()

    if ll.isEmpty():
        print(
            "[bold red]Data pemesanan kosong, tidak ada data yang bisa
dibatalkan![/bold red]"
        )

        input("\nTekan Enter untuk kembali...")
        continue

    while True:
        nama = input(
            "Nama customer yang akan dibatalkan [Enter untuk kembali]: "
        ).strip()

        if nama.strip().replace(" ", "").isalpha() or not nama:
            break

        print(
            "[bold red>Nama customer harus berupa huruf dan tidak boleh
berupa simbol.[/bold red]"
        )

        if not nama:
            _clear()
            print("Kembali ke menu...")
            continue

        _processing("Mencari data pemesanan...")

        # Cek duplikat di log_pemesanan (juga sumber kebenaran untuk
        # nama+film+sesi karena data ini permanent dan lengkap)
        duplicates = searchTempLogPemesananDuplicates(nama,
judul_film)

        # Tentukan jumlah_tiket & nomor_kursi yang akan dihapus
        # berdasarkan composite key
        target_jumlah_tiket = None

```

```

target_nomor_kursi = None
target_date = None

if len(duplicates) > 1:
    # Ada lebih dari 1 record dengan nama+film sama,
    # minta user pilih (mengikuti pola cancelQueue)
    print(
        f"\nTerdapat {len(duplicates)} data dengan nama '{nama}' "
        f"untuk film '{judul_film}':\n"
    )

    duplicate_list = list(duplicates.items())

    for idx, (log_id, data) in enumerate(duplicate_list, start=1):
        kursi_str = ", ".join(seat_sort(data.get("nomor_kursi", [])))
        print(
            f"{idx}. Jumlah tiket : {data.get('jumlah_tiket')}, "
            f"kursi : {kursi_str}, "
            f"tanggal : {data.get('date')} "
        )

    print("\n0. Kembali")

    while True:
        pilihan = input(
            "\nPilih nomor data yang akan dibatalkan: "
        ).strip()

        if not pilihan:
            _clear()
            print("Kembali ke menu...")
            break

        try:
            pilihan = int(pilihan)
        except ValueError:
            print("[bold red]Pilihan tidak valid![/bold red]")
            continue

        if pilihan == 0:

```

```

        _clear()
        print("Kembali ke menu...")
        break

    if 1 <= pilihan <= len(duplicate_list):
        selected_log_id, selected_data = duplicate_list[pilihan - 1]
        target_jumlah_tiket = int(
            selected_data.get("jumlah_tiket", 0)
        )
        target_nomor_kursi = [
            k.strip()
            for k in selected_data.get("nomor_kursi", [])
            if k and k.strip()
        ]
        target_date = selected_data.get("date")
        break

    print(
        f"[bold red]Pilihan harus 0-{len(duplicate_list)}![/bold red]"
    )

    # Jika user membatalkan (input kosong / 0)
    if target_jumlah_tiket is None:
        continue

    elif len(duplicates) == 1:
        # Tepat 1 record, gunakan composite key-nya
        selected_data = list(duplicates.values())[0]
        target_jumlah_tiket = int(selected_data.get("jumlah_tiket", 0))
        target_nomor_kursi = [
            k.strip()
            for k in selected_data.get("nomor_kursi", [])
            if k and k.strip()
        ]
        target_date = selected_data.get("date")
    else:
        # Tidak ada di log_pemesanan (mungkin hanya di temp/tiket
aktif)

        # Tetap lanjut dengan None agar deleteTicketByComposite coba
        # hapus berdasarkan nama saja

```



```

pass

# Hapus dari linked list menggunakan composite key
# (jika composite key tidak ada, fallback ke nama saja)
if target_jumlah_tiket is not None and target_nomor_kursi is not
None:
    refunded_ticket, refunded_seat = ll.deleteTicketByComposite(
        nama, target_jumlah_tiket, target_nomor_kursi
    )
else:
    refunded_ticket, refunded_seat = ll.deleteTicket(nama)

if refunded_ticket is None:
    _clear()
    print("[bold red>Nama tidak ditemukan.[/bold red]")
    input("\nTekan Enter untuk kembali...")
    continue

# Kembalikan kuota tiket & kursi yang sudah dibatalkan
available_ticket += refunded_ticket

available_seat.extend(refunded_seat)

available_seat = seat_sort(available_seat)

# Rewrite temp_seat agar refund bersifat persisten
# antar sesi
temp_seat = getAllData("temp_seat")

temp_seat[film_id] = {
    "judul_film": judul_film,
    "available_seat": available_seat,
}

updateData(data_dict=temp_seat, data_name="temp_seat")

# Simpan state ticket terbaru
_save_tickets(judul_film, ll)

# Hapus juga dari log_pemesanan agar laporan akurat,

```

```

# menggunakan composite key lengkap
deleteLogPemesananByComposite(
    nama=nama,
    judul_film=judul_film,
    jumlah_tiket=target_jumlah_tiket,
    nomor_kursi=target_nomor_kursi,
)

_processing("Membatalkan pemesanan...")

_clear()

print("[green]Pemesanan berhasil dibatalkan![/green]")

case "7. Cari Data Pemesanan":
    _clear()

    if ll.isEmpty():
        print(
            "[bold red]Data pemesanan kosong, tidak ada data yang bisa  
dicari![/bold red]"
        )

        input("\nTekan Enter untuk kembali...")
        continue

    while True:
        nama = input(
            "Nama customer yang akan dicari (Enter untuk kembali): "
        ).strip()

        if nama.strip().replace(" ", "").isalpha() or not nama:
            break

        print(
            "[bold red>Nama customer harus berupa huruf dan tidak boleh  
berupa simbol.[/bold red]"
        )

    if not nama:

```

```

        _clear()
        print("Kembali ke menu...")
        continue

    _clear()

    if not ll.searchTicket(nama):
        print("[bold red>Nama tidak ditemukan.[/bold red]")

    input("\nTekan Enter untuk kembali...")

case "8. Reset Antrean dan Pemesanan":
    _processing("Mereset...")

    resetOrder(queue=q, ticket=ll)

    # delete seat data at temp_seat and reset seat availability
    deleteTempSeat(judul_film)
    available_ticket = int(film_dict[film_id]["kuota_penonton"])
    available_seat = [
        "K" + str(i)
        for i in range(1, int(film_dict[film_id]["kuota_penonton"]) + 1)
    ]

    # Clear queue file for this film
    data = getAllData("data_antrean")

    if film_id in data:
        del data[film_id]
        updateData(data, "data_antrean")

    # Clear temp data pemesanan for this film
    deleteTempPemesanan(judul_film)

    _clear()

    print("[green]Antrean dan pemesanan berhasil di-reset.[/green]\n")

case "0. Kembali":
    _clear()

```

```
print("Kembali ke menu utama.")

break

case _:
    print("[bold red]Pilihan tidak valid![/bold red]\n")
```